

DCE — working with real documents, and changing the code

August 17, 2026

CONTENTS

DCE — working with real documents, and changing the code	
Part 1 — A new document arrives	
The triage order, and why it is this order	
Step 1 — Did the reader see the document at all?	
Step 2 — What did the classifier decide, and why?	
Step 3 — Only now, look at the registry	
Deciding what to do with a batch of new documents	
Adding documents to the corpus	
The rule on real documents	
Part 2 — The measurement discipline	
Before any change	
After the change	
Two traps that have already cost time here	
Read the numbers in this order	
Part 3 — Changing the registry	
3.1 Adding a new doctype	
The anchor rules, which the loader enforces	
Zones — the trap worth knowing before you use one	
The checklist	
3.2 A document that abstained	
3.3 A document classified as the WRONG doctype	

Part 4 — Configuration and reading problems

It came back 422 needs_ocr

The environment that matters

Latency, from your own first real run

Part 5 — Things that will bite you

Part 6 — The known limits

DCE — working with real documents, and changing the code

For: Nilendu · 17 Aug 2026

Everything here is checkable. Where a claim has a number behind it, the number is stated; where something is unmeasured, it says so. The commands are the ones actually used to build and verify this system, not illustrative.

Companion documents in the repo: `docs/ARCHITECTURE.md` (how it works),
`docs/RUNBOOK.md` (what to do when one document misbehaves).

Part 1 — A new document arrives

The triage order, and why it is this order

The single most common mistake is to reach for the registry when the problem is upstream. **Always answer these in order.** Each one rules out a whole class of cause.

STEP 1 — DID THE READER SEE THE DOCUMENT AT ALL?

```
python3 - <<'EOF'
from dce.ingest.pipeline import ingest
r = ingest(open('doc.pdf','rb').read())
print(r.status, r.text_source, 'truncated=', r.truncated, r.limits_hit)
print('reason:', r.reason or '(none)')
for p in (r.view.pages if r.view else []):
    print(f'  p{p.page}: alnum={p.alnum_chars:5d} adequate=
{p.text_adequate} '
          f'image={p.image_fraction:.2f} {p.width:.0f}x{p.height:.0f}')
EOF
```

Or open it in the console and read the **How each page was read** panel, which shows the same thing.

Stop here if: any page shows `adequate=False`, or `truncated=True` appears, or `limits_hit` is non-empty. The classifier never saw the whole document, and no registry change will fix that. Go to Part 4, “It came back needs_ocr or truncated”.

STEP 2 — WHAT DID THE CLASSIFIER DECIDE, AND WHY?

The console’s Analyze page shows the evidence, the four gates and the contenders. From logs:

```
docker logs dce 2>&1 | grep 'classify\.'
```

`classify.accept` gives you doctype, confidence, margin, coverage and the runner-up. `classify.abstain` gives you **which gate failed** and what it nearly was. That one line is usually the whole diagnosis.

STEP 3 — ONLY NOW, LOOK AT THE REGISTRY

And even then: change an **anchor**, not a threshold. See Part 3.

Deciding what to do with a batch of new documents

When a set of real documents arrives, sort them into four piles. The pile determines the work.

Pile	Symptom	What it means	Action
A	Correct doctype, good margin	working	add to corpus as a regression guard
B	Abstained	the cascade was not sure	Part 3.2 — usually a missing anchor
C	Wrong doctype	treat as urgent	Part 3.3 — usually an over-broad decisive anchor
D	422 <code>needs_ocr</code> / truncated	never reached the classifier	Part 4 — configuration or a reading problem

Pile C is the one that matters. A wrong doctype is a compliance incident; an abstention is safe and routes to a human. This asymmetry drives every design decision in the system and should drive your triage too.

Adding documents to the corpus

The corpus is what makes every later change measurable. A doctype with no corpus document is a claim nobody can check.

```
corpus/<cc>/<doctype_id>.pdf      # us | ca | mx
corpus/<cc>/manifest.jsonl         # one JSON line per document
```

```
{
  "file": "corpus/us/us_w9.pdf",
  "expected_doctype": "us_w9",
  "source_url": "https://www.irs.gov/pub/irs-pdf/fw9.pdf",
  "kind": "blank_form",
  "notes": "IRS official blank Form W-9, digital PDF with text layer."
}
```

Bundles go elsewhere. `corpus/bundles/` with `bundles.jsonl`, which declares page ranges per constituent. The filename is deliberately *not* `manifest.jsonl`, because `tools/corpus_test.py` globs `corpus/*/manifest.jsonl` and expects **one** doctype per file — a bundle there is scored as a wrong answer per bundle and quietly lowers the precision figure the whole project is judged on. Regenerate with `tools/make_bundles.py`.

If you add a test that walks corpus PDFs, exclude `corpus/bundles/`. One already had to be fixed for exactly this.

THE RULE ON REAL DOCUMENTS

Never commit a document containing a real person's data. Three files were deleted from this repo for carrying real individuals' identifiers. Use issuer specimens, synthetic samples, or a real layout with every value replaced — keeping the *printed wording* intact, since that is what the classifier reads.

If a document is too sensitive to commit, keep it outside the repo and measure locally. A measurement you can run is worth more than a corpus entry you cannot share.

Part 2 — The measurement discipline

This is the part that is not optional. Three attempts at improving this system have been made and reverted because they were measured and cost precision. The discipline is what caught them.

Before any change

```
# 1. BEFORE baseline – note the exact command; you will re-run it
verbatim
.venv/bin/python tools/corpus_test.py --ingest --url
http://localhost:8200 --verbose

# 2. Full test suite
.venv/bin/python -m pytest -q
```

After the change

```
# 3. REBUILD – the container mounts no source. A host edit does NOT reach
a running service.
docker compose build dce && docker compose up -d --force-recreate dce

# 4. Verify the rebuild actually took. This has caught a silent no-op
before.
docker exec dce python3 -c "import dce.ingest.pdf as m;
print(hasattr(m, 'PageVerdict'))"

# 5. Re-run the SAME command from step 1
```

Two traps that have already cost time here

`--ingest` is required. Without it the harness reads PDFs itself with its own PyMuPDF path and never exercises `dce/ingest/pdf.py`. A baseline captured without it measured nothing about an ingest change — the before and after were identical for the wrong reason.

The container mounts no source. Editing a file on the host changes nothing in a running container. Step 4 exists because that mistake produces a perfectly plausible “no change detected” result.

Read the numbers in this order

1. **Precision first** (`precision_when_answered`). It is the property. If it drops, revert — whatever else improved.
2. **Then wrong count.** Zero is the target and the current state.
3. **Then accuracy / abstention.** These are the ambition, not the guarantee.

Current baseline: **97 correct / 0 wrong / 20 abstained, 100% precision** on the 117-document corpus.

Part 3 — Changing the registry

3.1 Adding a new doctype

Doctypes live in `dce/registry/{usa,canada,mexico,crosscountry}.py`. A complete example, from the real registry:

```
DocTypeSpec(
    doctype_id="us_passport",
    label="US Passport (book)",
    country="US",
    category=Category.identity,
    issuing_authority="U.S. Department of State",
    officially_valid=True,          # counts as proof of identity
    for KYC
    anchors=[
        _a("P<USA", decisive=True, controls=Controls.MRZ_PREFIX),
        _a("United States of America", zone=Zone.title),
        _a("PASSPORT", zone=Zone.title),
        _a("Department of State"),
    ],
    id_patterns=[MRZ_TD3_USA],
    confusable_with={
        "us_passport_card": "the card is a TD1 wallet card whose MRZ
starts I<USA...",
        "ca_passport": "a Canadian book's MRZ starts P<CAN and its data
page is bilingual",
    },
    negative_anchors=["PASSPORT CARD", "PASSEPORT", "PASAPORTE"],
    fields=[...],
)
```

THE ANCHOR RULES, WHICH THE LOADER ENFORCES

A decisive anchor is a near-proof claim and must declare `controls` saying what justifies it. The loader refuses to import a pack that violates this.

Controls	value	Means	Strength
	FORM_NUMBER	a numbered form issued by one authority	strong
	CONTROL_NUMBER	a printed control number	strong
	MRZ_PREFIX	ICAO 9303 prefix, e.g. P<USA	strong
	STATUTE_TITLE	statutory title of an instrument	medium
	ISSUER_NAME	the issuing body's name	medium
	ISSUER_TEMPLATE	verbatim wording from the issuer's own template	medium
	CLASS_NAME_UNCONTESTED	known-weak — a class name nothing else currently collides with	weak

`CLASS_NAME_UNCONTESTED` is documented in the registry as *the tier of no evidence yet*. The loader forbids any other doctype from declaring the same string, so the pack fails to import the moment two collide. That failure is the design working.

Fuzzy matching is refused for decisive anchors. A near-proof claim must be *seen*, not approximated.

ZONES — THE TRAP WORTH KNOWING BEFORE YOU USE ONE

A `zone=Zone.title` anchor can only fire on a reading that **has** zones. Only `azure_layout` produces them. A PDF text layer, local OCR and `azure_read` are all zone-free — every block is `Zone.body`.

Consequence, measured: `us_drivers_license`, `ca_drivers_license` and `ca_nexus` have **every** decisive anchor title-gated, so on any other reading they cannot be decisively identified at all. If you gate an anchor to a zone, you are deciding that doctype only works on DI.

THE CHECKLIST

1. Add the `DocTypeSpec` to the right pack.
2. Add a corpus document and a manifest line.

3. Add a registry test asserting it classifies.
4. **Run the full corpus.** A new doctype changes the IDF profile and can move documents that have nothing to do with it — this is not hypothetical, it is why the corpus run is mandatory rather than advisory.

3.2 A document that abstained

In order of likelihood:

1. **Zones.** If `text_source` is `native_text`, `local_ocr` or `azure_read`, no title-gated anchor could fire. Try `azure_layout` before touching anything.
2. **Coverage low** → the document's vocabulary barely overlaps the doctype's profile. The registry entry does not match this issuer's template. Add anchors from the *actual* wording.
3. **Margin small, right doctype leading** → two doctypes are genuinely confusable. Look at `runners_up`. The fix is a **discriminating anchor**, and an entry in `confusable_with` explaining the difference.

Do not lower a threshold. They are what hold precision at 100%. If you lower one, the corpus run must show precision held — and it usually will not.

3.3 A document classified as the WRONG doctype

Urgent. Work backwards:

1. Which anchor fired for the wrong doctype? The console's evidence panel names it.
2. If a **decisive** anchor fired wrongly, that anchor is the bug. Decisive means near-proof; if it is not, demote it to non-decisive or narrow its text.
3. Add the losing doctype to the winner's `negative_anchors` if a specific string should argue against it.
4. Record the distinction in `confusable_with` — that text is what a future reader needs.

Precedent: `PERMANENT RESIDENT CARD` was decisive for `us_green_card` and matched Canadian PR cards. The fix was demotion, not a threshold.

Then re-run the corpus. Anchors interact; a change that fixes one document routinely costs two others.

Part 4 — Configuration and reading problems

It came back 422 `needs_ocr`

Check in this order:

1. **Is a recogniser configured?** `/readyz` → `ocr.configured_providers`. Empty means no.
2. **Did the request decline it?** The reason says *"this request declined recognition"*. The console sends `local_ocr: false` when the read-channel toggle is on **lexical**, and that setting is **sticky in the URL** (`?read=lexical`) across reloads and across documents. Switch to `auto`.
3. **The prefix mistake.** `AZURE_DI_ENDPOINT` configures the *extraction tiers*. Ingest OCR needs `DCE_INGEST_AZURE_DI_ENDPOINT`. Symptom: T2/T3 work, the OCR picker says "not configured here".

```
docker exec dce env | grep -i azure | sed -E 's/(KEY|SECRET|TOKEN)=.*\/\1=<redacted>/I' | sort
```

4. Reachability from inside the container:

```
docker exec dce python3 -c "import
urllib.request;print(urllib.request.urlopen('http://YOUR-
HOST:5000',timeout=5).status)"
```

The environment that matters

```
DCE_INGEST_OCR_SERVICE_ENABLED=true
DCE_INGEST_OCR_SERVICE_PROVIDER=azure_layout      # prefer over
azure_read: Read has no zones
DCE_INGEST_OCR_SERVICE_TRUST_BOUNDARY=on_premises # declared, never
inferred
DCE_INGEST_AZURE_DI_ENDPOINT=http://your-host:5000
DCE_INGEST_AZURE_DI_KEY=...
DCE_INGEST_AZURE_READ_ENDPOINT=http://your-host:5000
DCE_INGEST_AZURE_READ_KEY=...
DCE_INGEST_TEXT_LAYER_POLICY=verify                # trust | verify |
always_ocr
```

Boot refusals — these fail loudly by design: both local and service enabled with no `OCR_DEFAULT_PROVIDER`; a trust boundary that is not exactly `external` or `on_premises` (`on-premises` with a hyphen fails); `OCR_SERVICE_PROVIDER=azure_di` (it is `azure_layout`); `TEXT_LAYER_POLICY=always_ocr` with no recogniser.

Quoting: `VAR="true"` in a `--env-file` arrives as the literal `"true"` with quotes. Never quote in an env-file.

Latency, from your own first real run

Your internal DI took **9.1 seconds and 16 polls for a single page**, against a 30 s timeout and a 60-poll cap. About 3× headroom for one page. Nobody has measured whether that scales linearly — a 10-page scan is the test to run. Both bounds are environment variables:

```
DCE_INGEST_OCR_SERVICE_TIMEOUT_SECONDS=30
DCE_INGEST_OCR_SERVICE_MAX_POLLS=60
```

A T3 extraction timeout is different and is hardcoded at 120 s (`dce/extract/azure_specialist.py`, `_POLL_DEADLINE_SECONDS`). No environment variable raises it. Note also that a client-side timeout does **not** cancel the Azure job — the pages may still be billed.

Part 5 — Things that will bite you

Trap	Symptom	Fix
<code>?read=lexical</code> sticky in the URL	everything returns <code>needs_ocr</code>	switch to <code>auto</code>
<code>AZURE_DI_ENDPOINT</code> vs <code>DCE_INGEST_AZURE_DI_ENDPOINT</code>	picker greyed out, T2/T3 fine	use the prefixed one
Container mounts no source	change appears to do nothing	rebuild
<code>--ingest</code> omitted from the harness	before/after identical	always pass it
Quoted values in an env-file	flag ignored	never quote
Zone-gated anchor on a non-DI path	photo ID abstains	use <code>azure_layout</code>
Bundles in <code>corpus/<cc>/manifest.jsonl</code>	precision figure drops	use <code>corpus/bundles/</code>
<code>get_texttrace()</code> for invisible text	interpreter aborts after ~130 pages	never use it

That last one deserves the emphasis: on PyMuPDF 1.28.0 it over-decrements `Py_None` once per span per call and dies with an **uncatchable SIGABRT** — no traceback, kills in-flight requests. Use `page.read_contents()` with a `Tr` regex if you ever need the signal.

Part 6 — The known limits

State these plainly rather than discovering them under pressure.

Segmentation recall is 35%. Precision is 100% — every split it proposes is real — but it misses seams it cannot see. By shape:

Bundle shape	Recall
same size / all text / no form number	0%
mixed size / all text / no anchors	36%
same size / text + scan	50%
mixed size / all text / with anchors	75%

Two same-size typed documents with no form number are **structurally indistinguishable**. No threshold reveals a signal that is not there; raising recall needs a *new* signal, and the obvious one — per-page classification — was measured at 71.3% precision and rejected.

Two documents on ONE page are not separated. Segmentation is page-level: a boundary is a page number. A W-9 and a bank statement side by side on one sheet returns **one confident answer** and never mentions the second. Fixing it needs sub-page region analysis — buildable, since DI returns paragraph bounding boxes, but a new capability rather than a tuning change.

Three photo-ID doctypes need `azure_layout`, `us_drivers_license`, `ca_drivers_license`, `ca_nexus` — every decisive anchor title-gated.

The corpus has no real bundles and no mixed documents. Every segmentation number is measured on synthetic joins and long regulatory filings, which is the least favourable material available. This is the largest open question and only real documents close it.